

CS 4341 Assignment 2

Code due 2026-09-23; oral reports 2026-09-24

1 Overview

In this assignment, you will build an agent that plays Cylindrical Connect Four. Choose a suitable Chapter 5 approach, such as minimax, alpha-beta pruning, iterative deepening, Monte Carlo tree search, or another approach you can explain and justify. You will decide how your agent judges positions and moves, then explain your design and evaluate its results in a five-minute oral report.

There is no single expected algorithm, evaluation method, or move. The tests check only the common requirements: your agent chooses legal moves on time, handles terminal states correctly, and can finish a game. Your design choices will matter in the class tournament, but tournament results do not affect your grade.

2 Cylindrical Connect Four

The game uses the usual seven-column by six-row Connect Four board. Red moves first. On each turn, choose a column that is not full; the piece falls into that column's lowest empty cell. Players may not pass.

The key change is that the board's left and right edges meet. Horizontal and diagonal lines can wrap across this seam, while vertical lines do not wrap. For example, columns 6, 7, 1, and 2 are consecutive. Four consecutive pieces belonging to one player win. A full board with no winner is a draw. Use `cylindrical_connect_four.py` as the authority for these rules instead of copying them into `main.py`.

Because the board wraps, there is no permanently special center or edge column. In particular, a fixed center-column bonus is not symmetric on this board. A strategy can still create an asymmetric position through its earlier moves.

The supplied state contains the complete board and the player whose turn is next. Facts such as immediate wins, blocks, playable threats, forks, and groups that cross the seam are features you may calculate from that state; they are not separate game states.

3 Starter Files

Download and extract the starter bundle. It contains five files; keep them together in one directory. Make your changes only in `main.py`. The other files provide the game, shared interface, public tests, and a way to play against your agent.

3.1 `main.py`

Set the following constant and complete the required function in `main.py`:

1. `GROUP_NAME`
2. `adversarial_search(problem, state) -> Action | None`

Set `GROUP_NAME` to the nonempty name your group wants displayed in the tournament.

`adversarial_search` receives the game problem and the current state. It must:

- choose a move for `problem.to_move(state)`;
- return one of the legal moves from `problem.actions(state)`;
- return `None` if the state is terminal or has no legal moves;
- use the supplied problem methods to explore the game rather than copying the rules into `main.py`; and
- return in less than 10 seconds on every call.

How your agent chooses among legal moves is up to you. You might use a heuristic evaluation, quiescence search or selective extensions, rollouts to terminal states, MCTS policy or value estimates, or another justified Chapter 5 variant. These ideas can be combined. For example, a heuristic may be a helper used by depth-limited search, but there is no required heuristic function or signature. Put every helper, class, and setting your agent needs in `main.py`, and use only Python's standard library.

The 10-second limit is a hard maximum, not a target. Reaching it counts as resignation in a tournament game. If your search uses a clock, stop early—for example, after 8–9 seconds—to leave room for normal timing variation and cleanup. You choose the depth or iteration limit, rollout or selection policy, exploration settings, caching, randomization, and tie-breaking.

3.2 `adversarial_search.py`

The provided `adversarial_search.py` file defines the interface your search must use. `AdversarialSearchProblem` provides:

- `initial`, the problem's starting state;
- `to_move(state)`, the player whose turn it is;
- `actions(state)`, the legal moves;
- `result(state, action)`, the state after a legal move;
- `is_terminal(state)`, whether the game is over; and
- `utility(state, player)`, the exact terminal value from the given player's point of view.

Use these methods instead of reimplementing the game rules or redefining the interface in `main.py`.

3.3 `cylindrical_connect_four.py`

The provided `cylindrical_connect_four.py` file contains the game rules and state representation. States cannot be changed after they are created and may be used as dictionary keys. The most useful state members are:

- `state.player`, either "red" or "yellow", for the next player to move;
- `state.columns`, the seven columns stored from bottom to top;
- `state.cell(column, row)`, which reads one zero-based board position;
- `state.ply`, the number of pieces played;
- `state.winner`, the winning player or `None`; and
- `state.is_full`, whether the board is full.

The Python code numbers columns 0 through 6; the terminal display labels the same columns 1 through 7. Always choose from `problem.actions(state)` instead of assuming a column is open. `WINDOWS` lists every possible four-cell winning line, including wrapped lines, and `iter_window_cells(state)` provides their contents for position analysis. Printing a state, or calling `format_state(state)`, shows a dotted seam and repeats columns 1 through 3 on the right so that wrapped lines remain visible.

3.4 `play.py`

After implementing your agent, play against it from the starter directory:

```
python play.py
python play.py --human yellow
```

The first command makes you Red. The second makes you Yellow, so the agent moves first. Enter a displayed column number from 1 through 7, or `q` to quit. The display uses `X` for Red, `0` for Yellow, and `.` for an empty cell. It also checks the agent's move and reports how long the decision took. Do not submit `play.py`.

3.5 `test_cases.py`

Run the public tests from the starter directory:

```
python -m pytest -q test_cases.py
```

If Python reports that `pytest` is missing, install it once with `python -m pip install pytest`. Your submitted `main.py` must still use only Python's standard library.

The tests check `GROUP_NAME`, legal moves, the 10-second limit, terminal-state handling, and completion of a game against a seeded random opponent. The private grader checks your search on five different fixed positions. Passing the public tests shows that your code follows the required interface; it does not measure tournament strength.

4 Tournament

Submissions that pass every automated test will enter a single-elimination bracket. Each match has ten games from the empty board, with each agent playing five times as Red and five times as Yellow. A timeout, exception, process failure, or illegal move counts as a resignation for that game. A win is worth one game point, and a draw is worth half a point.

If the match is tied after ten games, the agent with the lower total decision time advances. The tournament seed breaks an exact tie. The tournament records moves, response times, resignations, results, and bracket advancement. Tournament results do not change your assignment grade.

5 Submission

This is a group project for 2–4 students. Submit through Canvas: open Assignment 2 there, and Canvas will take you to its Gradescope grader. Form your full group before submitting. One group member should upload a single top-level file named exactly `main.py` on behalf of the group. Do not upload a ZIP or any other files. Keep the names `GROUP_NAME` and `adversarial_search`, and do not change the required function signature. Your oral report will be given separately in class.

6 Oral Report

Your group will give its oral report in class on September 24. The five-minute limit is strict, and you may use the board. Your explanation must match your submitted code. Focus on the decisions and evidence that best explain your agent; do not attempt a line-by-line code reading. Address all five topics below within the time limit.

1. **Frame the problem and your approach.** Briefly explain the agent's goal, the cylindrical rule that most affects its decisions, the interface and timing constraints, and your overall strategy.
2. **Explain and justify the search.**
 - Identify the Chapter 5 method or variant you chose and explain why it fits this problem.
 - State what the search optimizes and whose perspective its values represent.
 - Explain how and when the search stops, including when it reaches a terminal state.
 - Describe its depth, pruning, selection, exploration, extensions, or rollout policy as appropriate.
3. **Explain how your agent judges positions and moves.**
 - Explain how your method compares nonterminal states or possible moves. This may involve a heuristic, quiescence or extensions, terminal rollouts, MCTS policy or value estimates, or another method.
 - Identify the state information your method uses and explain why it should help.
 - Explain how the cylindrical seam affects your analysis or move selection.
 - Identify important game information your method leaves out or approximates.
4. **Explain the most important implementation choices.**
 - Describe how `adversarial_search` explores the game tree and chooses a move.
 - Identify any important data structures, caching, tie-breaking, or randomization.
 - Explain how the code uses the problem interface and handles terminal states and legal moves.
 - Explain how your internal budget keeps every call below 10 seconds.
5. **Evaluate the finished agent with evidence.**
 - Report relevant public-test or game results and decision times.
 - Explain what the results show about correctness and strategy quality.
 - Identify the most important limitations or tradeoffs in your design.

7 Grading

7.1 Submission and Correctness: 40%

The automated grader reports all 40 points in this section. Oral-report points are recorded separately by the instructor.

- **File naming: 2%.** Your code must be a top-level file named `main.py`.
- **Tournament group name: 3%.** Replace `GROUP_NAME` with a nonempty group name.
- **Search test cases: 10%.** There are five private states. Returning a timely legal action on each state is worth 2%.
- **Terminal-state handling: 5%.** Return `None` for a terminal state within the time limit.
- **Complete game: 20%.** Complete a game against the random-move opponent with every action legal and below the per-move time limit.

Automated points measure compatibility and completion, not the strategic quality of a particular legal move.

7.2 Oral Report: 60%

The oral report is worth 60% and is divided into timing and technical-content components. Tournament results do not change the oral-report grade.

Time Limit: 20%

- Finish in 5 minutes or less: 20%.
- Exceed 5 minutes but finish in 7 minutes or less: 10%.
- Exceed 7 minutes: 0%.

Technical Content: 40%

Technical-content points require a clear, accurate explanation and a correct implementation that matches it.

- **Framing and overall strategy: 5%.** Clearly explain the objective, constraints, and general approach.
- **Adversarial-search design: 10%.** Correctly explain and justify the search method, optimized objective, player perspective, game-tree representation, terminal evaluation, and stopping or pruning strategy.
- **Position and move evaluation: 10%.** Clearly explain and justify how the agent compares nonterminal states or moves, which game information it uses, how it handles cylindrical wraparound, and what it leaves out or approximates.
- **Implementation decisions: 10%.** Clearly explain tree exploration, use of the required interfaces, important data structures, legal and terminal handling, tie-breaking or caching, and time management.
- **Results and evaluation: 5%.** Report precise evidence and accurately evaluate the agent's correctness, running time, strategy quality, and tradeoffs.